Capstone Project Report

On

ShopForHome

Submitted by:

Abhishek Yadav

Technology:

Angular + ASP.NET Core Web API

Date:

23 March 2026

Instructor:

Ramesh Nediyadath

Great Learning

DECLARATION

I, Abhishek Yadav, hereby declare that the project titled "ShopForHome – E-Commerce Application" submitted as part of the training program conducted by Great Learning for Wipro is my original work.

I confirm that this project has been developed by me and has not been submitted previously for any academic or professional purpose. All sources of information used in this project have been properly acknowledged.

I take full responsibility for the authenticity and correctness of the work presented in this report.

Date: 23 March 2026
Place: Darbhanga

Signature: ____
(Abhishek Yadav)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Wipro and Great Learning for providing me with the opportunity to work on this capstone project titled “ShopForHome”.

I would also like to thank the instructors and mentors for their continuous guidance, support, and valuable feedback throughout the training program. Their guidance helped me enhance my technical skills in full-stack development using Angular and ASP.NET Core.

Finally, I am thankful for the opportunity to apply my knowledge in a practical environment and gain hands-on experience in building a real-world application.

Date: 23 March 2026
Signature: ____
(Abhishek Yadav)

ABSTRACT

A web-based e-commerce program called "ShopForHome" was created to give consumers a smooth online shopping experience for home goods like lighting, furniture, and décor. The system's scalable and effective architecture is ensured by the use of Angular for the front end and ASP.NET Core Web API for the back end. Users can register, log in, browse products by category, filter products based on price and rating, and manage their shopping basket with this application. The user experience is improved with extra features like wishlist management and discount coupons.

From an administrator's point of view, the system offers features to monitor stock levels, create sales reports, upload products in bulk using CSV files, and manage users and products through CRUD operations. To guarantee effective inventory management, low stock level notifications are also put into place. Database design, API development, and front-end integration are all part of the project's organized development methodology. API testing and documentation are done with Swagger. Additionally, security mechanisms like authorization and authentication are used.

All things considered, the ShopForHome application exemplifies the real-world application of full-stack development utilizing cutting-edge technology and software engineering best practices.

TABLE OF CONTENTS

1. INTRODUCTION 1

2. SYSTEM ANALYSIS 2

3. REQUIREMENTS 3
   3.1 Functional Requirements 3
   3.2 Non-Functional Requirements 3

4. SYSTEM DESIGN 4
   4.1 System Architecture 6

5. DATABASE DESIGN 7
   5.1 Main Tables in the System 7
   5.2 Relationships Between Tables 10
   5.3 ER Diagram 11

6. SYSTEM MODULES AND IMPLEMENTATION 12

7. TESTING 15
   7.1 Types of Testing 15
   7.2 Sample Test Cases 16

8. RESULT 17
   8.1 OUTPUT SCREENS 17

9. CONCLUSION 27

10. FUTURE SCOPE 28

11. REFERENCES 28

1. INTRODUCTION

E-commerce's explosive rise has completely changed how consumers buy goods and services. Efficient, scalable, and user-friendly programs that offer smooth user experiences are required due to the growing demand for online retail platforms. The "ShopForHome" initiative is a web-based e-commerce service that specializes on home goods like lighting, furniture, and décor.

This project's primary goal is to create a full-stack web application that lets customers browse products, filter them according to different criteria, and effectively manage their shopping cart. The system also provides features like user authentication, wishlist management, and discount coupon integration to enhance user engagement.

The application provides total control over user and product administration from an administrative standpoint. CRUD procedures, bulk product uploads utilizing CSV files, stock level monitoring, and sales report generation are all possible for admin users. In order to guarantee efficient inventory management, the system also incorporates alerts for low stock levels.

Angular is used for the front end of the application, and ASP.NET Core Web API is used for the back end. In order to provide seamless connection between the database and the application, Entity Framework is utilized for database interaction. Security features like permission and authentication provide secure data handling, and Swagger is incorporated for API testing and documentation.

All things considered, the ShopForHome project offers a workable solution for online purchasing systems and exemplifies the application of contemporary web development technology.

2. SYSTEM ANALYSIS

Understanding system requirements and determining user and administrator needs are the main goals of system analysis, a crucial stage in the software development life cycle. With effective user interaction and administrative control, the ShopForHome program is intended to offer a comprehensive e-commerce solution for household goods.

The User Module and Admin Module are the two primary modules that make up the system. Users can register, log in, browse products by category, filter products by price and rating, add items to their basket, control the quantity in their cart, and apply discount coupons using the User Module. Additionally, it contains a wishlist function for storing items for later buying.

CRUD functions for managing users and products are provided by the Admin Module. Administrators may monitor product stock levels, upload products in bulk using CSV files, and get alerts when stock drops below a predetermined threshold. The system also allows administrators to manage discount coupons for particular users and create sales reports for a predetermined period of time.

With a responsive frontend created with Angular and a well-structured backend created using ASP.NET Core Web API, the system guarantees effective data management and communication. Swagger is used for API testing and documentation, while Entity Framework is utilized for database operations.

All things considered, the system is made to be scalable, safe, and easy to use, successfully meeting user and administrative needs.

3. REQUIREMENTS

The requirements of the ShopForHome system are categorized into Functional Requirements and Non-Functional Requirements.

3.1 Functional Requirements:

1. User Registration and Login
2. Product Browsing by Category
3. Product Filtering by Price, Rating, and Category
4. Add to Cart and Manage Cart Quantity
5. Wishlist Feature
6. Discount Coupon Application
7. Admin CRUD Operations for Users and Products
8. Bulk Product Upload using CSV
9. Stock Management with Low Stock Notification
10. Sales Report Generation

3.2 Non-Functional Requirements:

1. The system should be user-friendly and easy to navigate.
2. The application should be responsive across different devices.
3. The system should ensure data security using authentication and authorization.
4. The application should provide fast performance and efficient data processing.
5. The system should be scalable to handle a large number of users and products

4. SYSTEM DESIGN

The ShopForHome project's system design outlines the organization of the program and the ways in which various modules communicate with one another. The presentation layer, application/API layer, and database layer make up the project's three-layer architecture. The system will continue to be well-organized, scalable, and simple to maintain thanks to this architecture.

Angular is used in the development of the presentation layer. It is in charge of managing all client-side activities and presenting the user interface. Registration, login, product browsing, category filtering, cart addition, wishlist management, coupon application, and order placement are all available to users. The frontend interface allows admin users to manage stock, reports, goods, and users.

The ASP.NET Core Web API is used in the development of the application layer. It serves as a link between the database and the front end. This layer handles all business logic. The Angular application sends it requests, which it then processes, validates, and returns the necessary response. Various modules, including order history, coupon processing, cart operations, product administration, authentication, and sales reports, are handled by separate API controllers.

All of the application's persistent data is kept in the database layer. The database for the project is SQL Server, and the ORM tool is Entity Framework Core. Database operations like insert, update, delete, and retrieval are made easier by Entity Framework, which also aids in mapping database tables to C# model classes.
Users, products, categories, carts, wishlists, coupons, orders, and order items are all stored in the database.

The User Module and the Admin Module are the two main modules that make up the system. Customers can engage with the shopping system through the User Module. User registration, login, product browsing, filtering, cart management, wishlisting, using coupons, and placing orders are all included. CRUD operations for users and products, bulk product upload via CSV, stock monitoring, coupon assignment, and sales report production are among the administrative functions offered by the Admin Module.

When a user acts on the Angular frontend, the system's communication flow starts. An HTTP request is sent by the frontend to the relevant Web API endpoint. After processing the request and interacting with the database using Entity Framework, the ASP.NET Core backend sends the outcome back to the frontend. The UI is then updated appropriately by the frontend. Smooth communication between every component of the application is ensured by this interaction.

Modular design is also used in the ShopForHome initiative. The project is easy to test and expand because each key feature is implemented as a separate module. For instance, order processing is kept apart from discount functionality, and authentication logic is kept apart from product management. This enhances the readability and maintainability of the code.

The system design also takes security into account. To limit access to sensitive modules, authorization and authentication procedures are put in place. Only approved admin users have access to admin-only features like product management, report generation, and user administration. This aids in preserving application security and data integrity.

All things considered, ShopForHome's system design offers an organized and effective architecture for putting into practice a full-stack e-commerce platform. It guarantees appropriate division of responsibilities, seamless front-end and back-end communication, and dependable administration of user and administrative features.

4.1 System Architecture

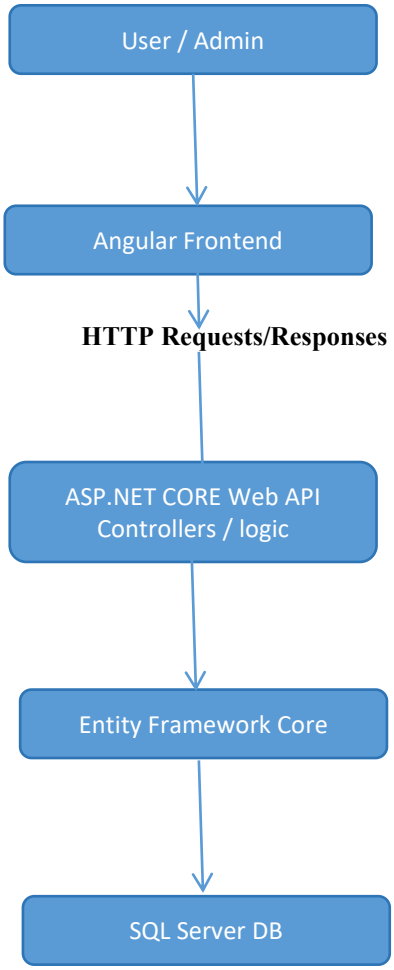


Fig: System Architecture of ShopForHome Application

The above diagram represents the system architecture of the ShopForHome application.

It follows a layered approach where the **Angular frontend** communicates with the **ASP.NET Core Web API** using HTTP requests.

The backend processes the data using **Entity Framework Cor**e and interacts with the **SQL Server** database. This layered architecture ensures scalability, maintainability, and efficient separation of concerns.

5. DATABASE DESIGN

The ShopForHome application's database design specifies how data is organized, kept, and controlled inside the system. The relational database management system used by the program is SQL Server, and database operations are handled by Entity Framework Core.

Primary and foreign keys are used to construct associations between the many tables that make up the database's relational model. This architecture enhances overall system performance, reduces redundancy, and guarantees data integrity.

5.1 Main Tables in the System

1. Users Table

Stores information about registered users.

Attributes:

- UserId (Primary Key)
- Name
- Email
- Password
- Role (Admin/User)

2. Categories Table

Stores product categories.

Attributes:

- CategoryId (Primary Key)
- Name
- Description

3. Products Table

Stores product details.

Attributes:

- ProductId (Primary Key)
- Name
- Price
- CategoryId (Foreign Key)
- Stock Quantity
- Description

4. CartItems Table

Stores items added to cart.

Attributes:

- CartItemId (Primary Key)
- UserId (Foreign Key)
- ProductId (Foreign Key)
- Quantity

5. Wishlist Table

Stores products added to the user's wishlist.

Attributes:

- WishlistId(Primary Key)
- UserId (Foreign Key)
- ProductId

6. Orders Table

Stores order details.

Attributes:

- OrderId (Primary Key)
- UserId (Foreign Key)
- OrderDate
- TotalAmount

7. OrderItems Table

Stores products in each order.

Attributes:

- OrderItemId (Primary Key)
- OrderId (Foreign Key)
- ProductId (Foreign Key)
- Quantity
- Price

8. Coupons Table

Stores discount Coupon details.

Attributes:

- CouponId (Primary Key)
- Code
- DiscountPercent
- ExpiryDate
- IsActive
- MinimumAmount

9. UserCoupon Table

Stores the relationship between users and coupons.

Attributes:

- UserCouponId (Primary Key)
- UserId (Foreign Key)
- CouponId

5.2 Relationships Between Tables

- A User can have multiple CartItems (1:N)
- Each CartItem is associated with one User and one Product (N:1)
- A User can have multiple Wishlist items (1:N)
- Each Wishlist item is linked to one User and one Product (N:1)
- A Product belongs to one Category, and a Category can have multiple Products (1:N)
- A User can place multiple Orders (1:N)
- Each Order belongs to one User (N:1)
- An Order can contain multiple OrderItems (1:N)
- Each OrderItem is associated with one Order and one Product (N:1)
- A Coupon can be applied to multiple Orders (1:N)
- A User can have multiple Coupons through UserCoupon table (M:N)

5.3 **ER Diagram:**

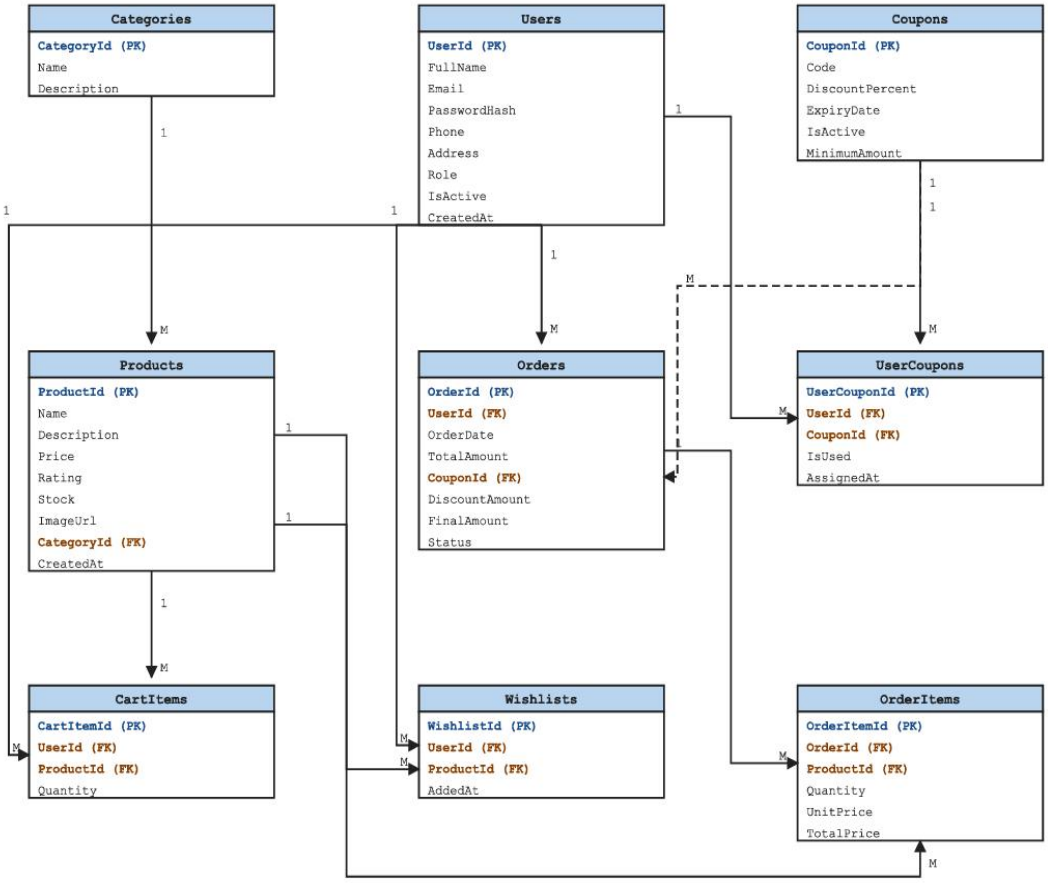


Fig: Entity Relationship Diagram of ShopForHome

6. SYSTEM MODULES AND IMPLEMENTATION

The ShopForHome application is divided into multiple modules to ensure better organization, scalability, and maintainability. Each module handles specific functionality of the system such as user management, product handling, and order processing.

1. User Module

This module manages user registration, login, and authentication. Users can create accounts, log in securely, and access system features based on their roles (Admin/User).

Features:

- User Registration
- User Login (JWT Authentication)
- Role Management(Admin/User)

2. Product Module

This module handles product management. Admin users can add, update, and delete products, while users can view and browse products.

Features:

- Add/Edit/Delete Products
- View Product List
- Product Details Display

3. Category Module

This module manages product categories. It helps in organizing products into different groups.

Features:

- Create Categories
- Update Categories
- Assign Products to Categories

4. Cart Module

This module allows users to add products to their cart before purchasing.

Features:

- Add to Cart
- Update Quantity

- Remove from Cart

5. Wishlist Module

This module allows users to save products for future reference.

Features:

- Add to Wishlist
- Remove from Wishlist

6. Order Module

This module manages order placement and tracking.

Features:

- Place Order
- View Order History
- Order Status Tracking
-

7. Coupon Module

This module manages discount coupons used during checkout.

Features:

- Apply Coupon
- Validate Coupon
- Track Coupon Usage

6.1 API ENDPOINTS

The backend APIs are developed using ASP.NET Core Web API. Swagger is used for testing and documentation of APIs.

The system provides RESTful APIs for different modules such as authentication, product management, cart operations, orders, and coupons.

Sample APIs:

Authentication:

- POST /api/auth/login → User login
- POST /api/auth/register → User registration

Products:

- GET /api/products → Fetch all products

- GET /api/products/{id} → Get product details
- POST /api/products → Add new product (Admin)
- PUT /api/products/{id} → Update product

Cart:
- GET /api/cart → Get user cart
- POST /api/cart → Add item to cart
- DELETE /api/cart/{id} → Remove item from cart

Orders:
- POST /api/orders → Place order
- GET /api/orders → Get user orders

Wishlist:
- POST /api/wishlist → Add to wishlist
- DELETE /api/wishlist/{id} → Remove from wishlist

Coupons:
- GET /api/coupons → Get available coupons
- POST /api/coupons/apply → Apply coupon

7.TESTING

To make sure that every module functions properly and the system acts as intended, testing is a crucial part of the ShopForHome project. Manual testing and API testing tools were used to test the application at the frontend and backend levels.

Swagger UI was used for backend API testing. Valid and incorrect inputs were used to test several APIs, including login, product management, cart operations, order placement, wishlist processing, and coupon validation. This aided in the verification of error management, response generation, and request handling.

Frontend testing involved running the Angular application and examining how users registered, logged in, browsed products, added items to their carts, managed wishlists, applied coupons, and viewed order histories. Additionally examined were admin-side features like low stock visibility and product management.

The testing process confirmed that the modules were integrated properly and the system produced the expected output.

7.1 Types of Testing

1. Unit/Module Testing
Each module such as User, Product, Cart, Order, Coupon, and Wishlist was tested individually.

2. Integration Testing
The interaction between Angular frontend, ASP.NET Core Web API backend, and SQL Server database was tested.

3. API Testing
Swagger UI was used to test API endpoints and verify JSON responses.

4. Functional Testing
All major functionalities such as login, registration, add to cart, place order, and coupon application were tested.

5. UI Testing
The frontend pages were checked for proper layout, navigation, and responsiveness.

7.2 Sample Test Cases

1. User Registration

Input: Valid name, email, password

Expected Result: User account created successfully

Actual Result: Passed

2. User Login

Input: Valid email and password

Expected Result: Login successful and token generated

Actual Result: Passed

3. Add Product to Cart

Input: Select product and click Add to Cart

Expected Result: Product added to cart successfully

Actual Result: Passed

4. Apply Coupon

Input: Valid coupon code

Expected Result: Discount applied successfully

Actual Result: Passed

5. Place Order

Input: Checkout with cart items

Expected Result: Order placed and stored in database

Actual Result: Passed

6. Wishlist

Input: Add/remove product from wishlist

Expected Result: Wishlist updated successfully

Actual Result: Passed

7. Admin Product Management

Input: Add/Edit/Delete product

Expected Result: Product updated in database and UI

Actual Result: Passed

8. RESULT

The ShopForHome project's finished product shows how a full-stack e-commerce platform may be implemented successfully. Both admin-side and user-side features are supported by the system. In addition to registering and logging in, users can explore products, add items to their carts, manage wishlists, apply coupons, and place orders. Products, categories, coupons, and stock information can all be managed by admin users.

Swagger was used to test the backend APIs, and Entity Framework Core was used to successfully integrate the application with SQL Server. The Angular-developed frontend successfully interacts with the backend APIs to generate the desired outcomes.

8.1 OUTPUT SCREENS

◆ Login Page

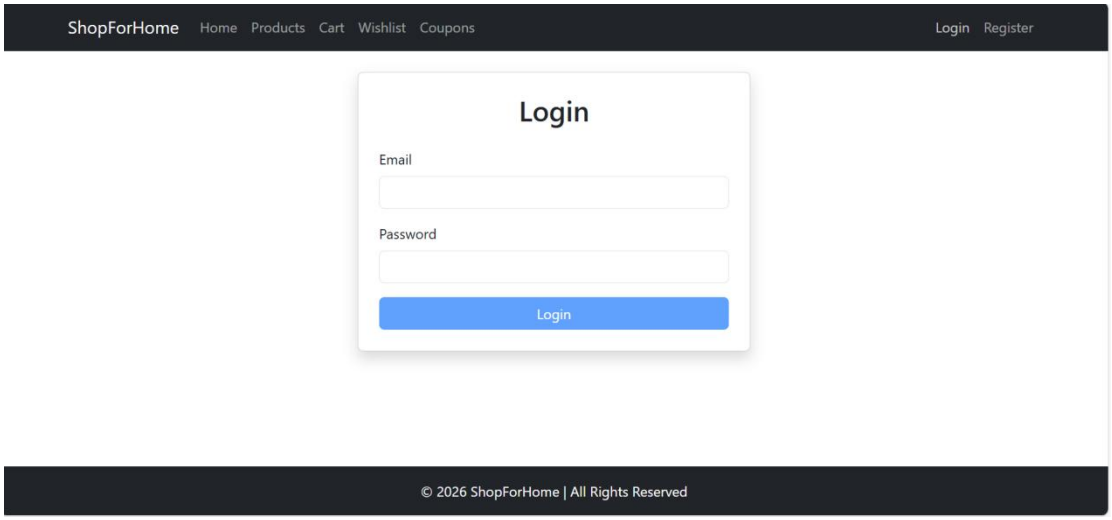


Fig: Login Page

◆ Register Page

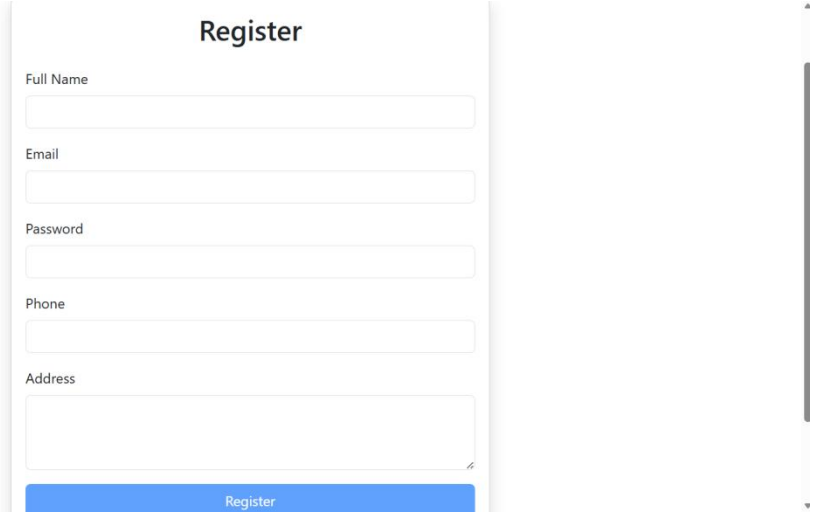
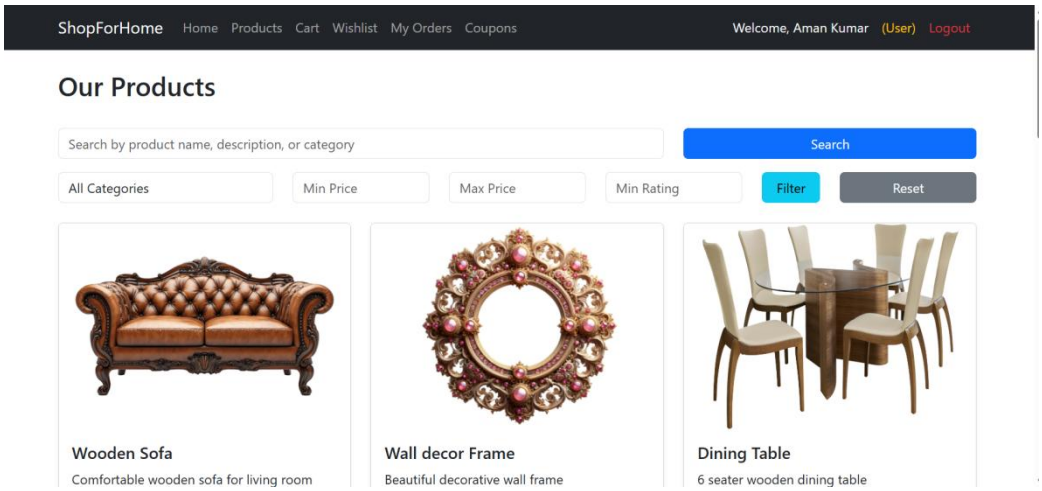


Fig: Register Page

◆ Product Listing Page



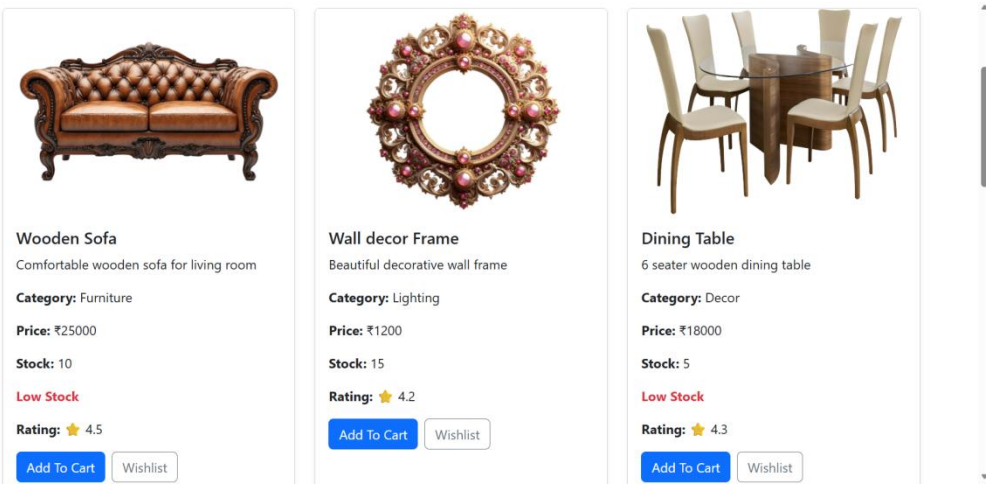
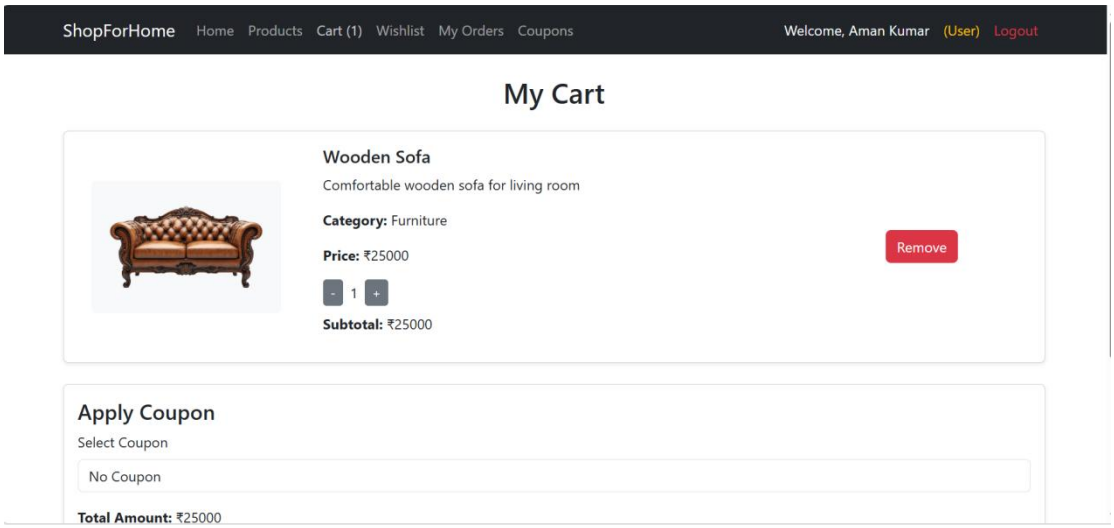


Fig: Product Listing Page

◆ Cart page



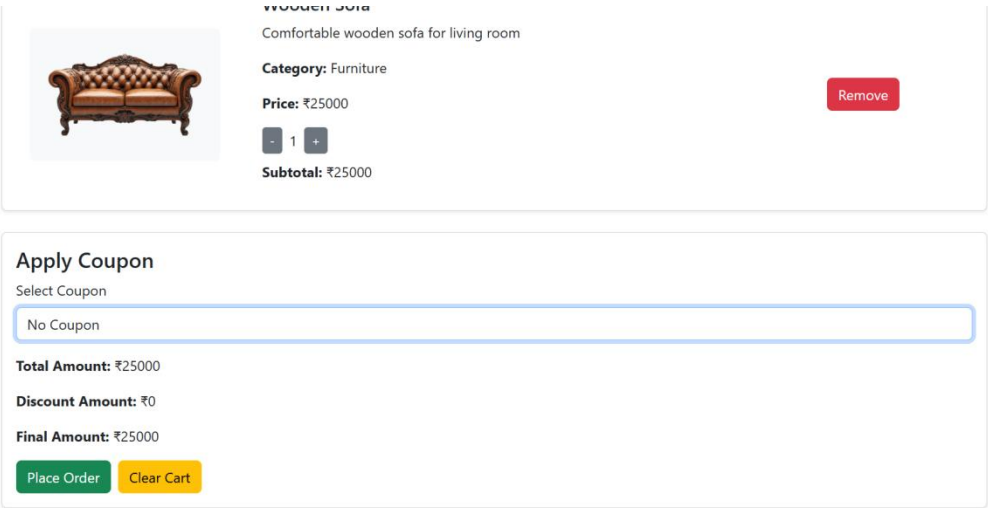


Fig: Cart Page

◆ Wishlist Page

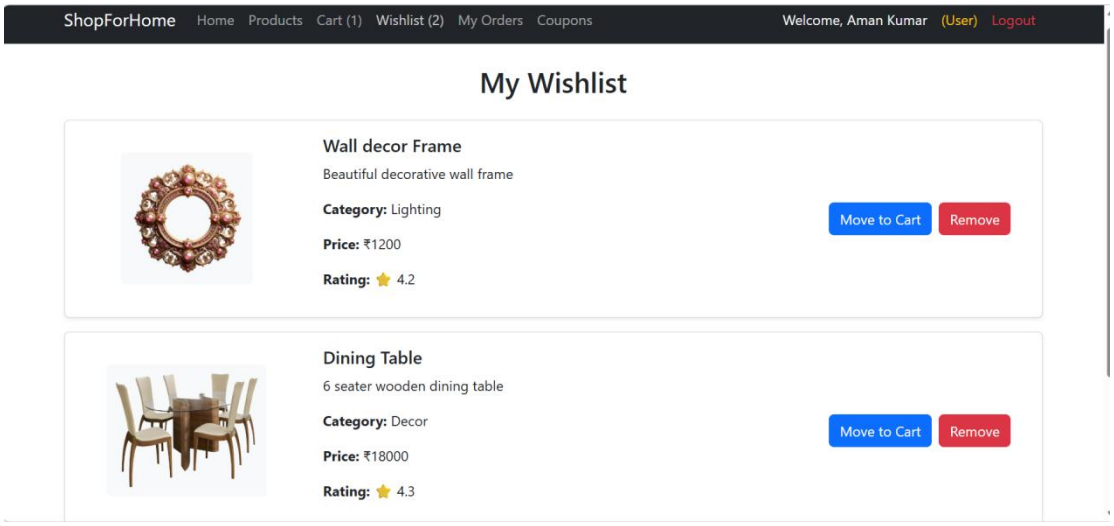


Fig: Wishlist Page

◆ Order History

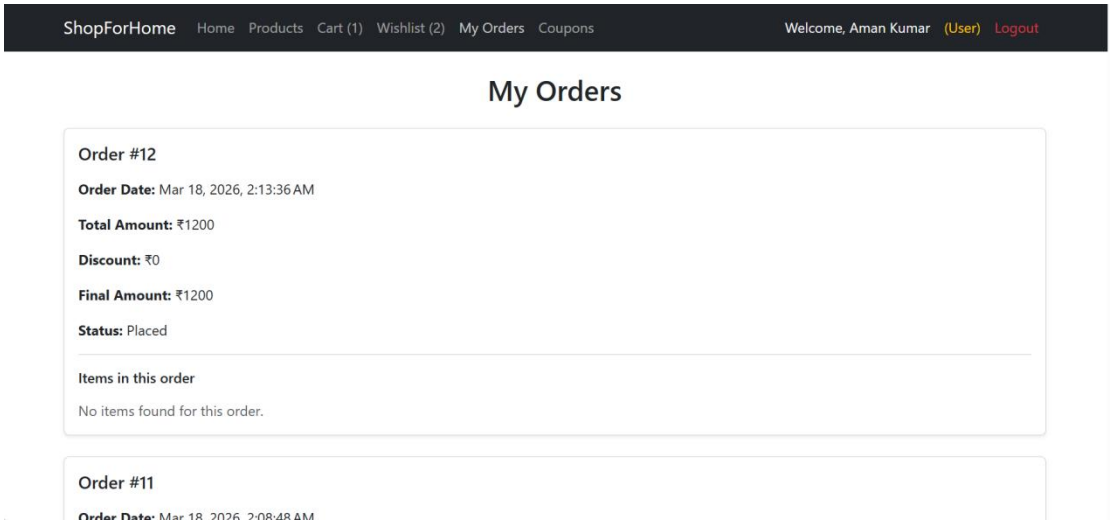


Fig:Order History Page

◆ User Coupon Page

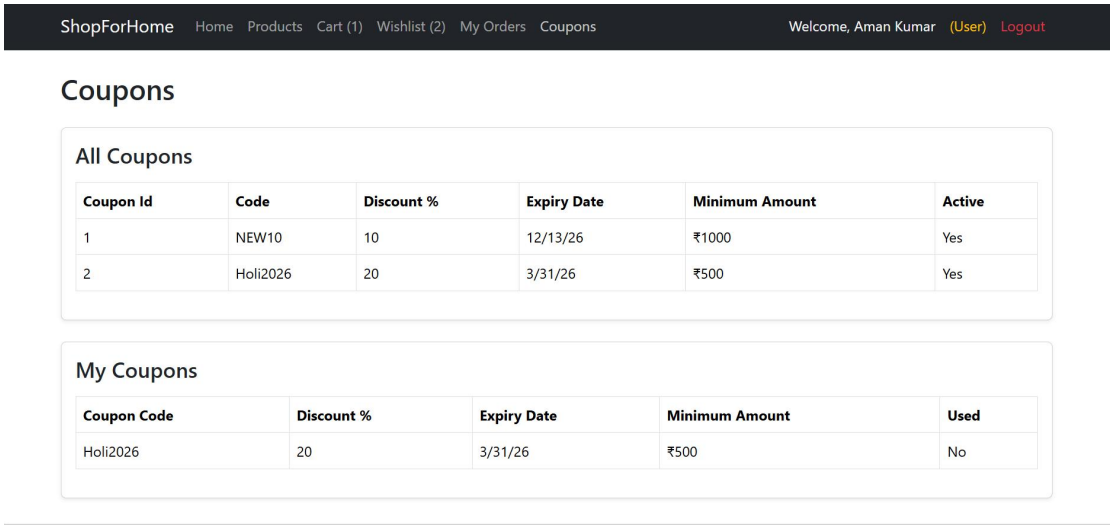


Fig: User Coupon Page

◆ Admin Product Management

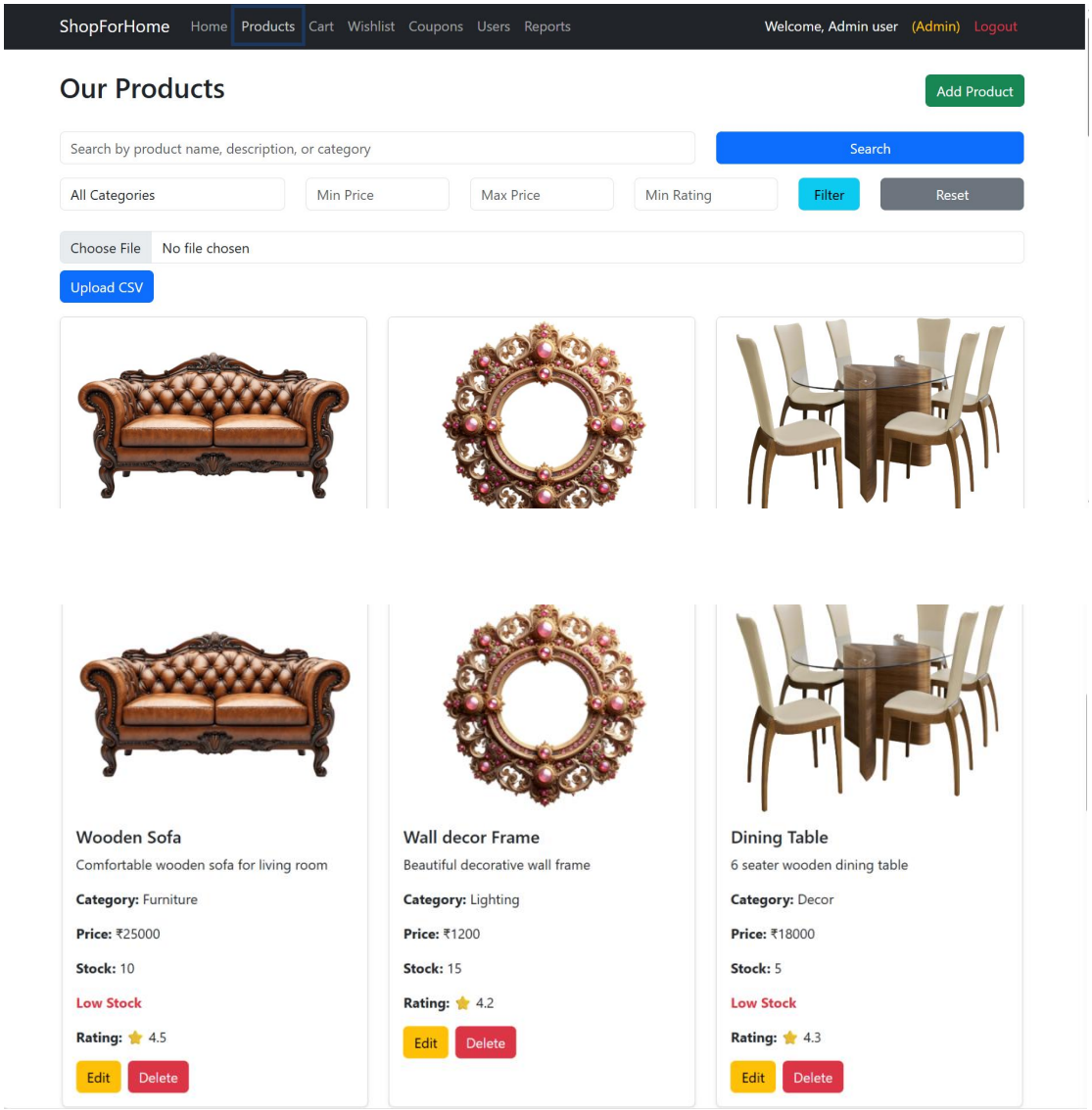
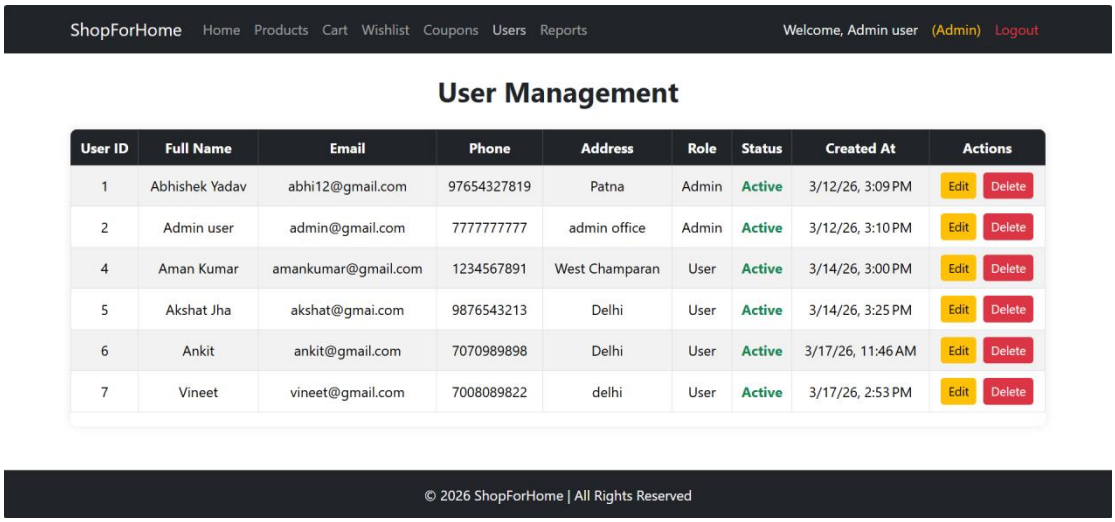


Fig: Admin Product Management Page

◆ Admin User Management



User ID	Full Name	Email	Phone	Address	Role	Status	Created At	Actions
1	Abhishek Yadav	abhi12@gmail.com	97654327819	Patna	Admin	Active	3/12/26, 3:09 PM	Edit Delete
2	Admin user	admin@gmail.com	7777777777	admin office	Admin	Active	3/12/26, 3:10 PM	Edit Delete
4	Aman Kumar	amankumar@gmail.com	1234567891	West Champaran	User	Active	3/14/26, 3:00 PM	Edit Delete
5	Akshat Jha	akshat@gmai.com	9876543213	Delhi	User	Active	3/14/26, 3:25 PM	Edit Delete
6	Ankit	ankit@gmail.com	7070989898	Delhi	User	Active	3/17/26, 11:46 AM	Edit Delete
7	Vineet	vineet@gmail.com	7008089822	delhi	User	Active	3/17/26, 2:53 PM	Edit Delete

Fig: Admin User Management Page

◆ Admin Coupon Page

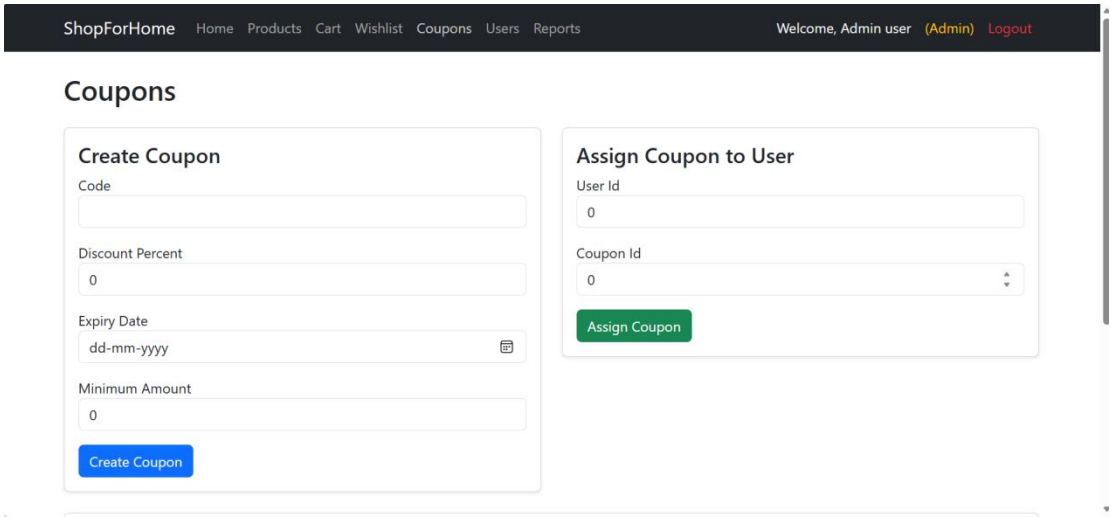


Fig: Admin Coupn Page

◆ Admin Sale Report and Low stock Display Page

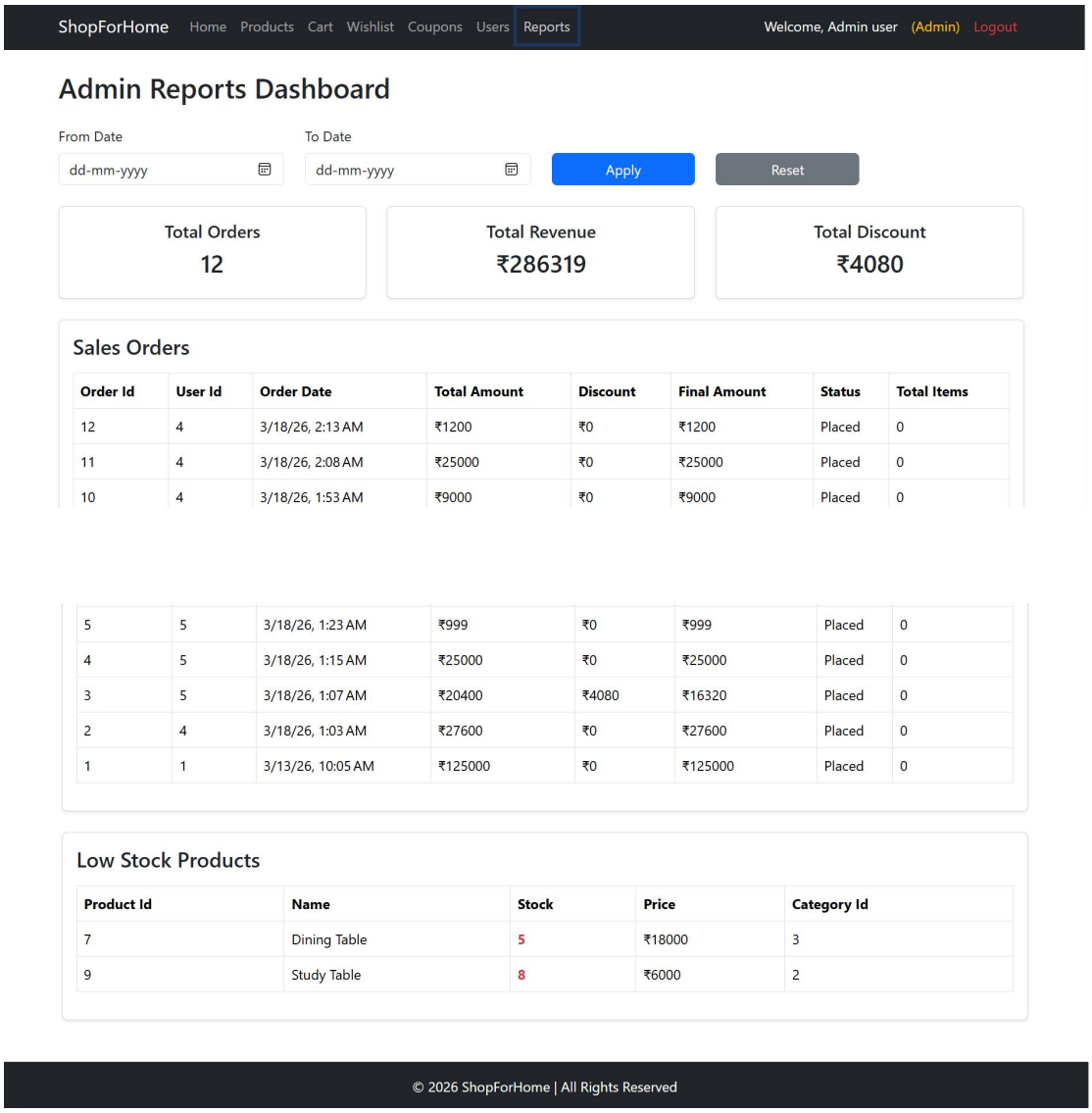
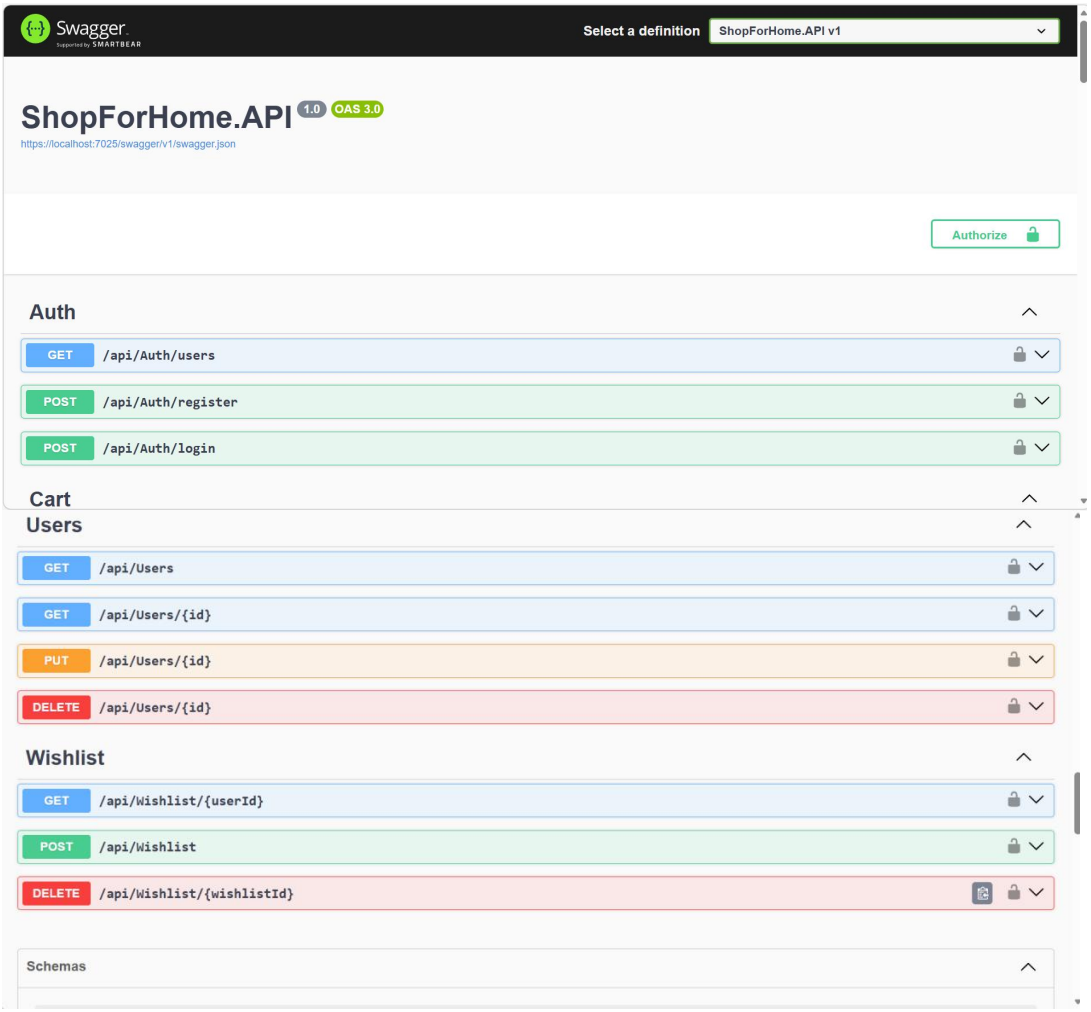


Fig: Admin Sale Report and Low stock Display Page

Swagger API Testing



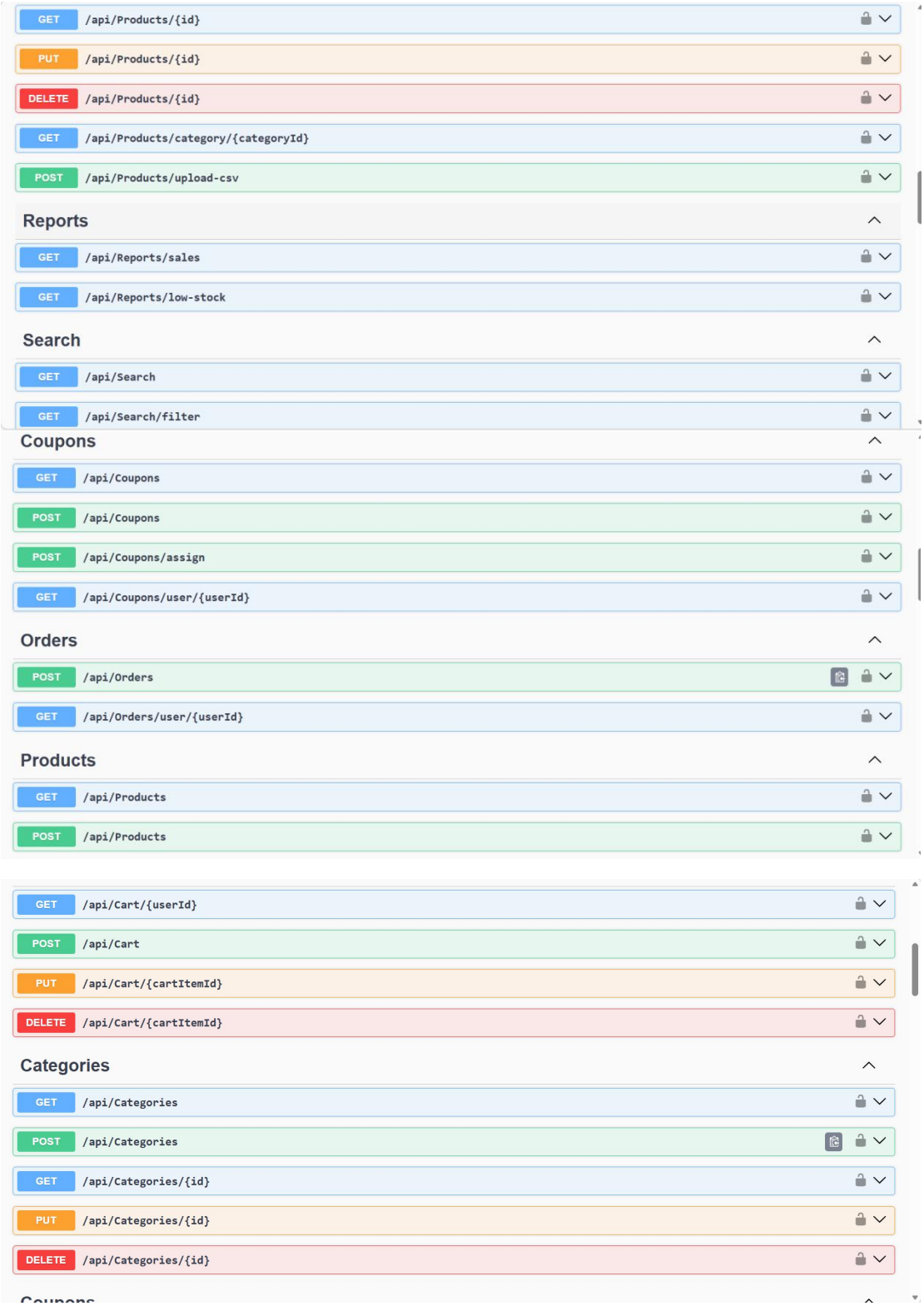


Fig: Swagger API Testing

9. CONCLUSION

The ShopForHome project effectively shows how to use Angular, ASP.NET Core Web API, Entity Framework Core, and SQL Server to create a full-stack e-commerce web application. User identification, product administration, cart operations, wishlist management, discount application, and order processing are among the essential features offered by the application.

Scalability, maintainability, and effective performance are guaranteed by the system's modular and layered architecture. RESTful APIs were used to integrate the frontend and backend, while Swagger was utilized for API validation and testing.

Database design, API development, frontend-backend integration, testing, and other aspects of real-world application development were all made clearer by this project. All things considered, the project achieves the goals of creating a dependable and user-friendly online commerce platform.

10. FUTURE SCOPE:

Advanced features can be added to the ShopForHome application to boost system capabilities and user experience. Real-time transactions can be made possible by integrating online payment platforms like Stripe or Razorpay.

It is possible to add other features like order tracking, email notifications, and delivery status updates. Additionally, the system may incorporate machine learning-based product reviews, ratings, and recommendation algorithms.

For increased scalability and accessibility, the application can be deployed on cloud platforms like Azure or AWS and tailored for mobile devices. To enhance business insights, admin dashboards with analytics and reporting capabilities might be included.

11. REFERENCES

1. Microsoft Docs – ASP.NET Core Web API
2. Microsoft Docs – Entity Framework Core
3. Angular Official Documentation
4. Microsoft Docs – SQL Server
5. Swagger Documentation